



Diplomatura en python para la ciencia de datos y machine learning

Módulo III: Deep Learning y TensorFlow

3. MPL - Funciones de Activación y Pérdida

Fórmula del Perceptrón

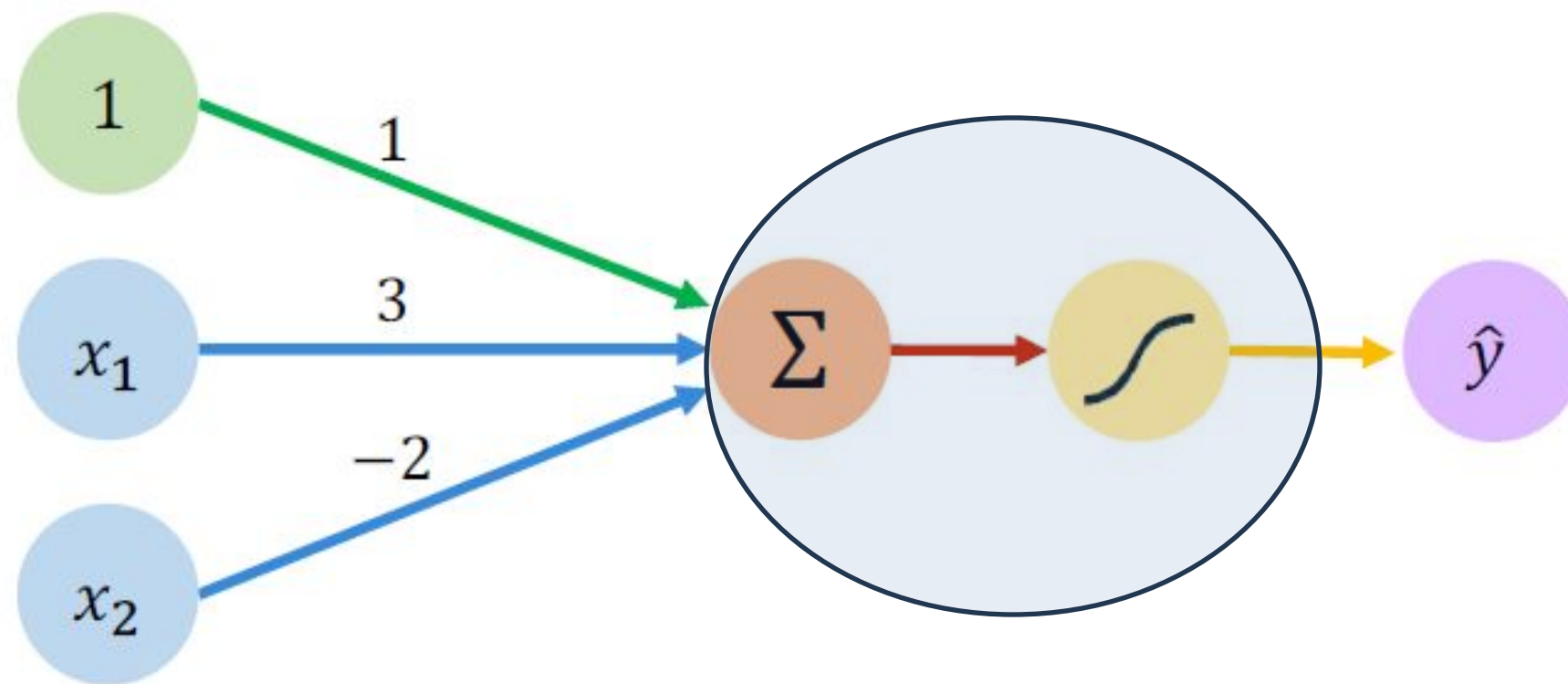
$$z = w_1x_1 + w_2x_2 + b$$

$$\hat{y} = g(z)$$

donde:

- x_1, x_2 : entradas,
- w_1, w_2 : pesos,
- b : bias,
- z : combinación lineal de las entradas,
- $g(z)$: función de activación (por ejemplo, escalón, sigmoide, ReLU, etc.),
- $\hat{y}$: salida o predicción del perceptrón.

Ejemplo de Perceptrón

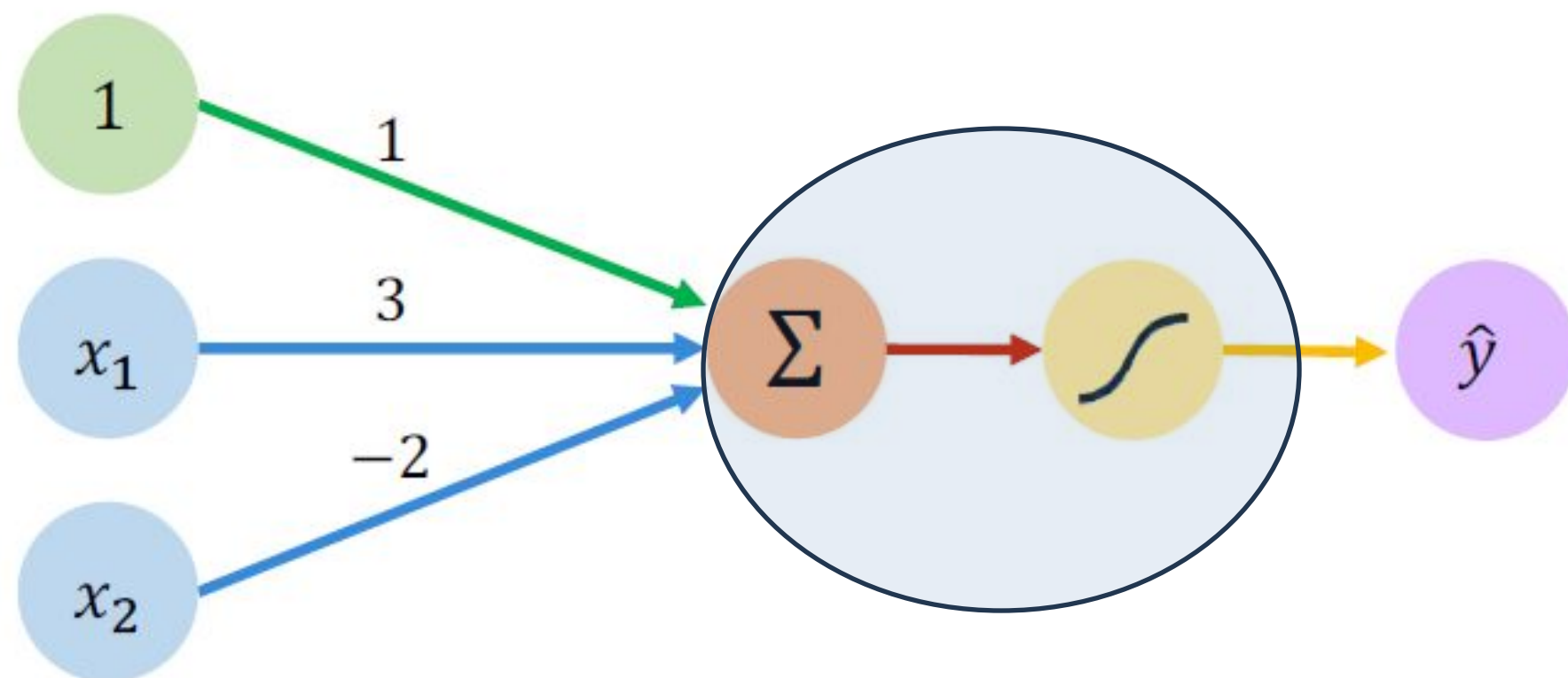


We have: $w_0 = 1$ and $\mathbf{W} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

This is just a line in 2D!

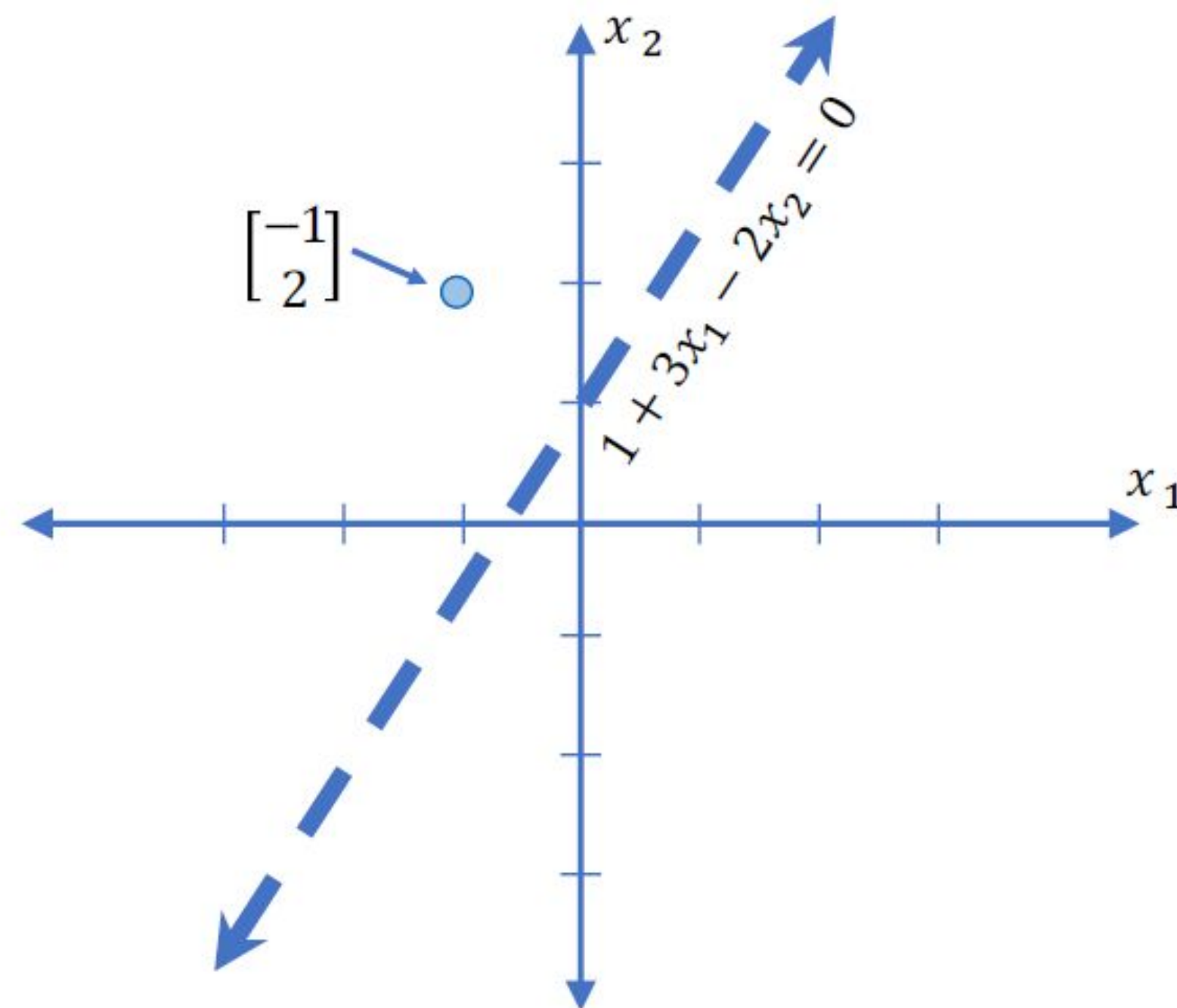
Ejemplo de Perceptrón



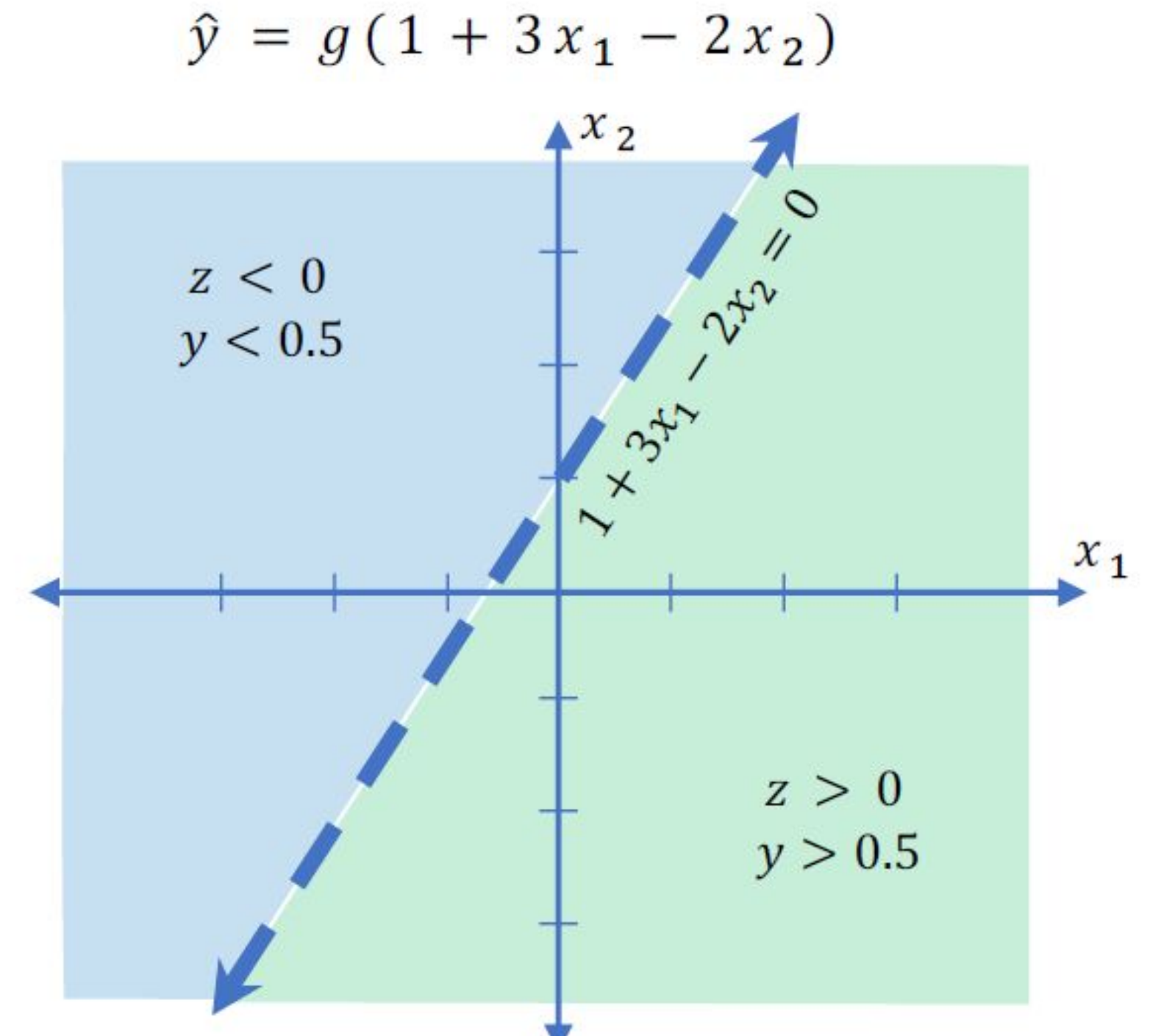
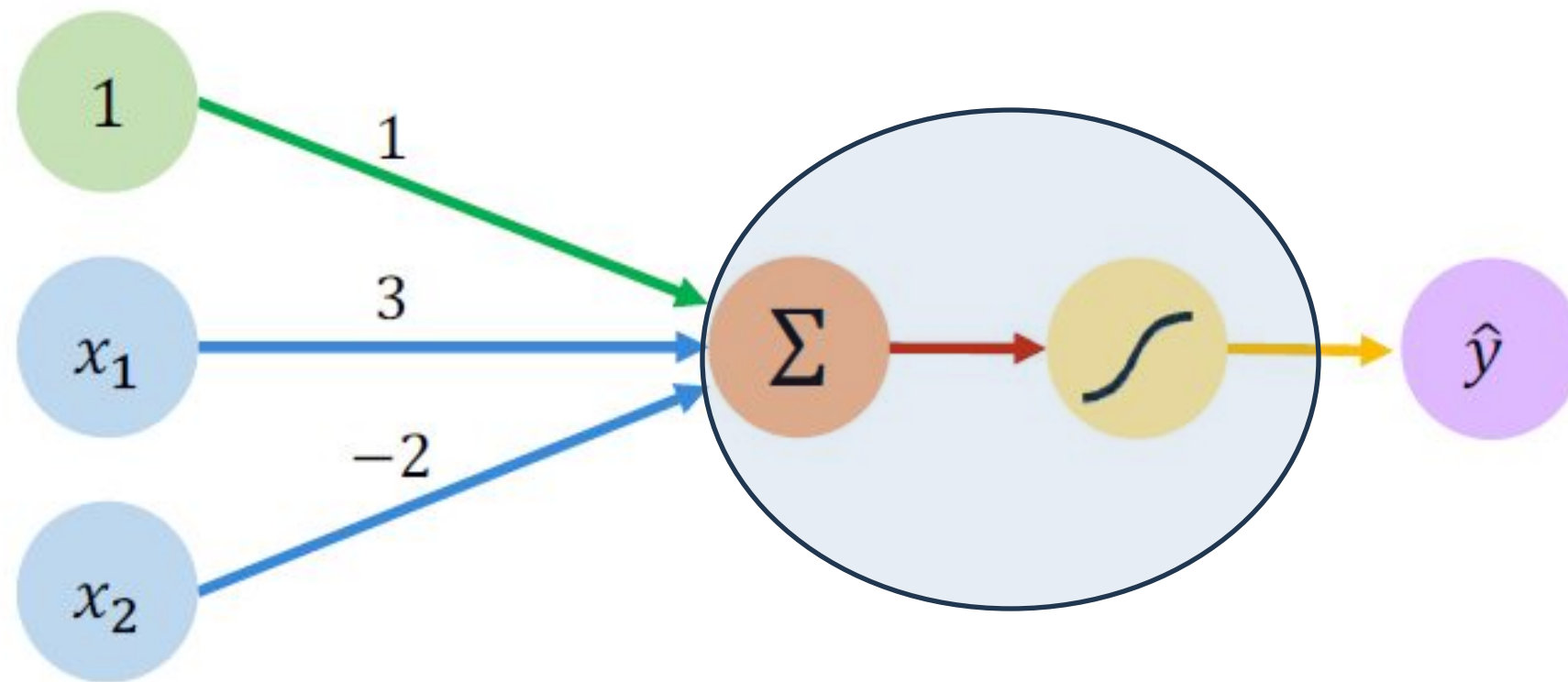
Assume we have input: $\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

$$\begin{aligned}\hat{y} &= g(1 + (3 * -1) - (2 * 2)) \\ &= g(-6) \approx 0.002\end{aligned}$$

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



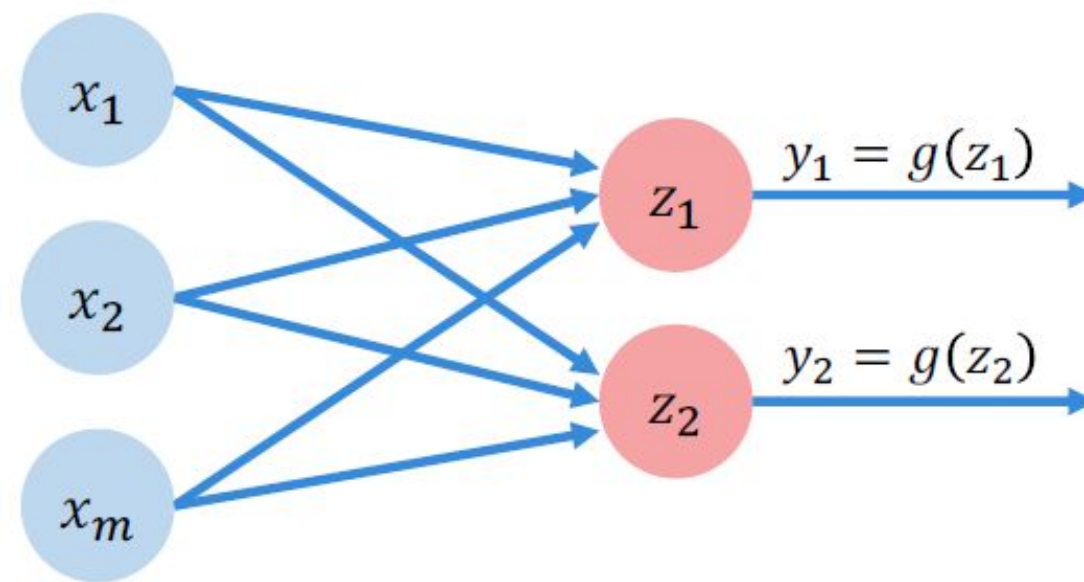
Ejemplo de Perceptrón



Perceptrón de múltiples Capas (MLP)

Al **apilar múltiples perceptrones**, obtenemos **redes neuronales**:

- Las primeras capas aprenden características simples.
- Las últimas capas combinan esas características para generar decisiones complejas.



Cuando queremos más de una salida, simplemente usamos varios perceptrones en paralelo:

$$y_1 = g(W_1 x + b_1)$$

$$y_2 = g(W_2 x + b_2)$$

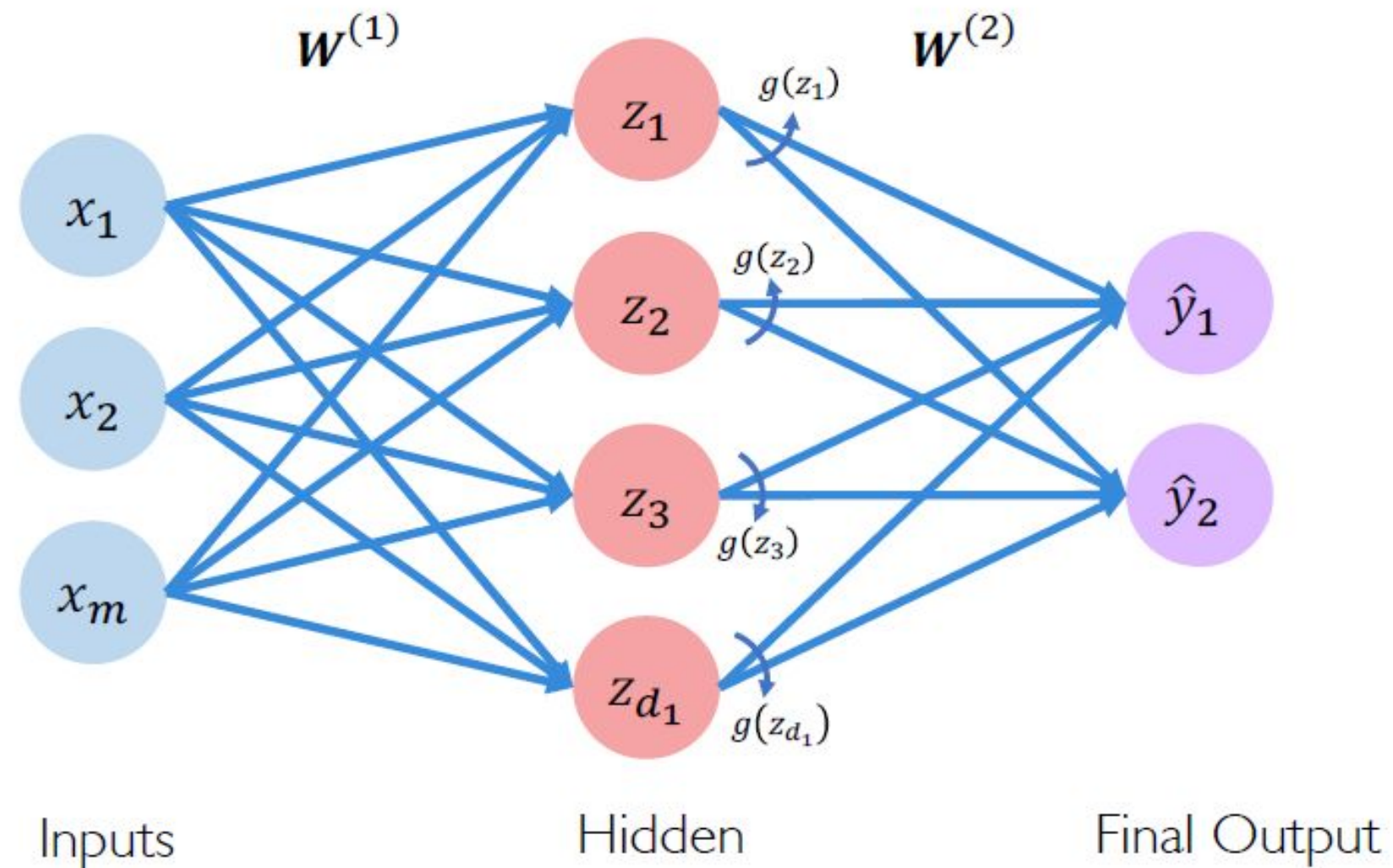
Esto permite que la red **prediga múltiples valores o clases simultáneamente**.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Red Neuronal de una Capa Oculta

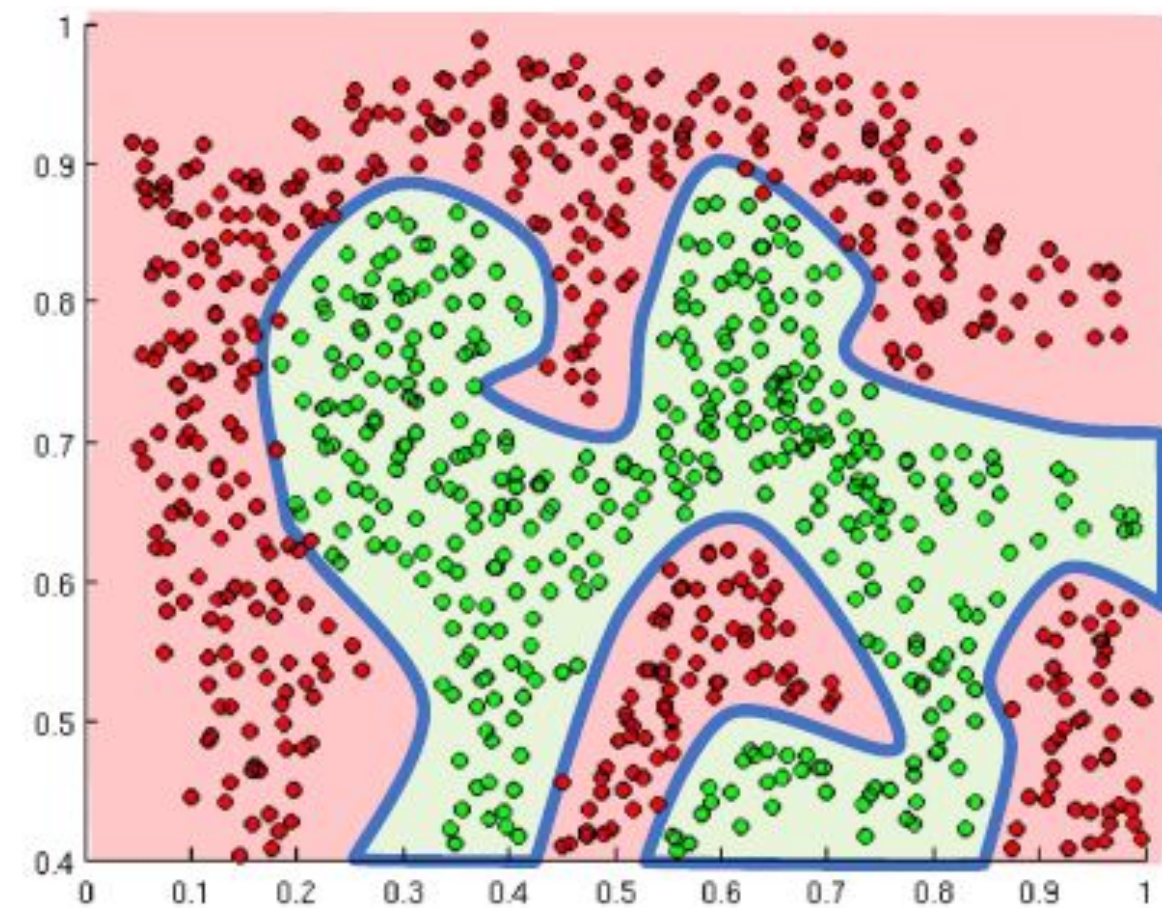
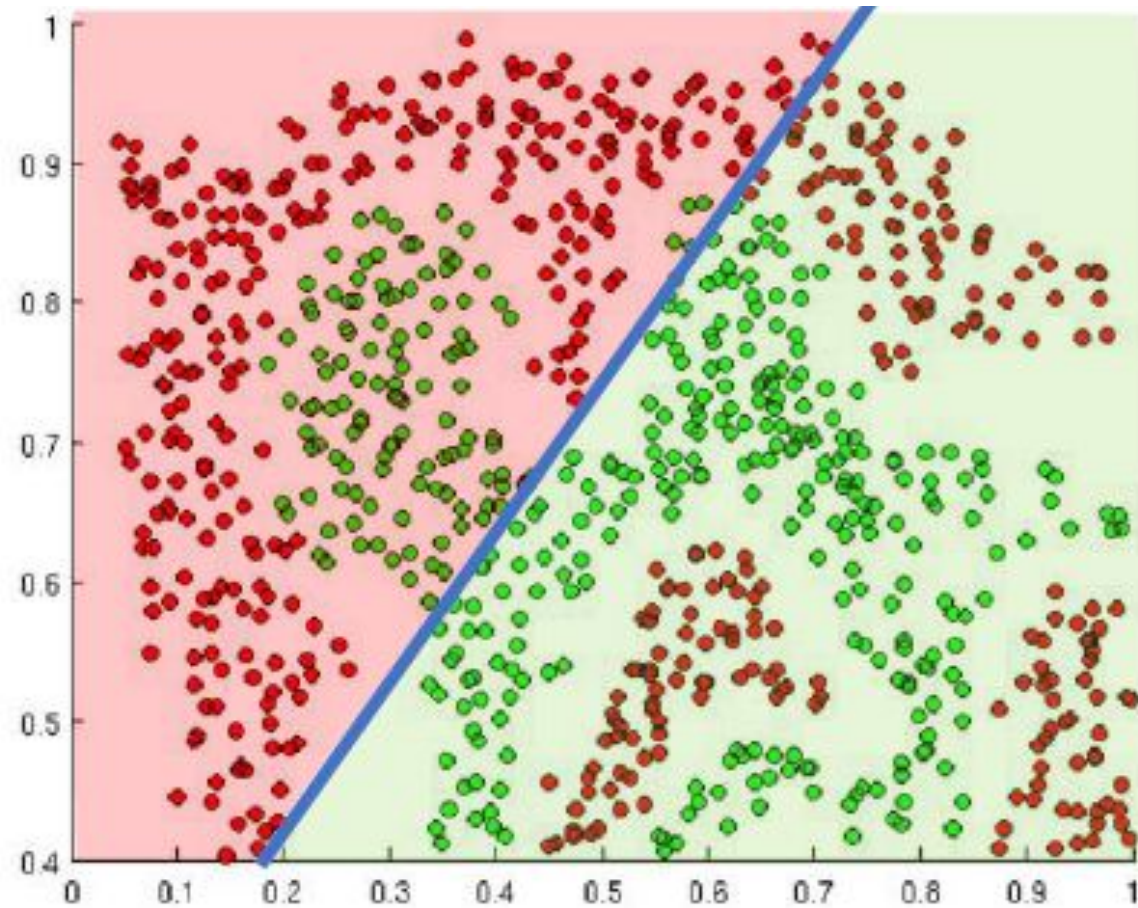
Estructura típica:

- Capa de entrada: variables $x_1, x_2, \dots, x_n$
- Capa oculta: perceptrones intermedios
- Capa de salida: genera las predicciones finales



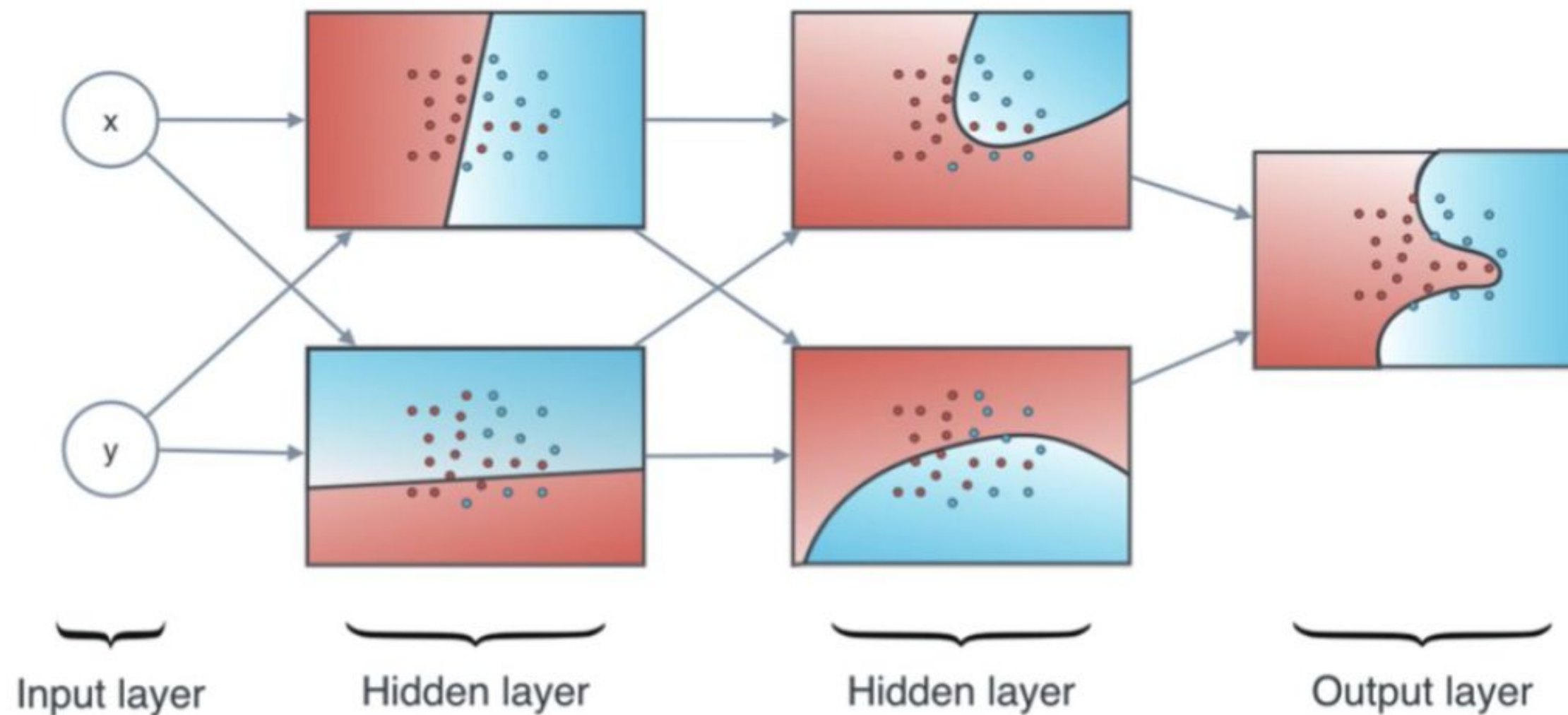
La Importancia de las funciones de Activación

Si usáramos solo **funciones lineales**, sin importar cuántas capas tuviéramos, el resultado seguiría siendo una **función lineal**.
Las no linealidades permiten que las redes neuronales aproximen funciones **altamente complejas**.



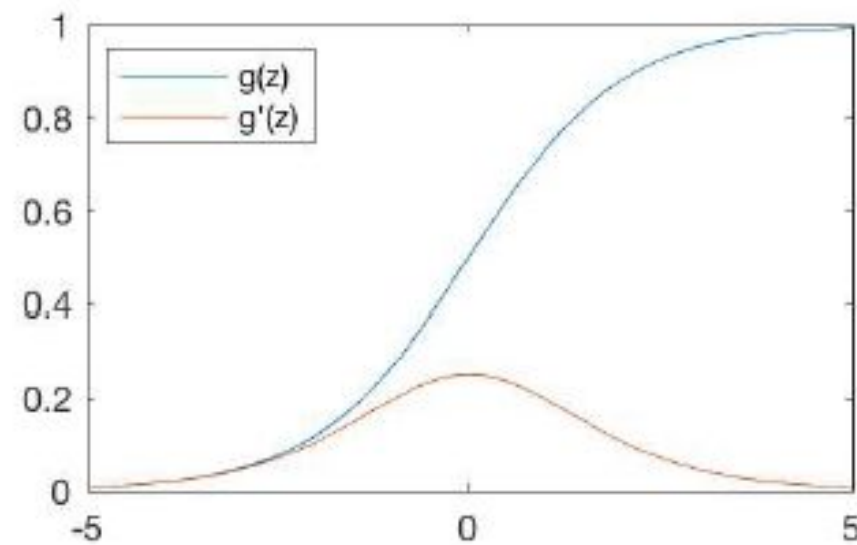
La Importancia de las funciones de Activación

Que nuestras funciones sufran una distorsión para romper la linealidad



Funciones de Activación Comunes

Sigmoid Function

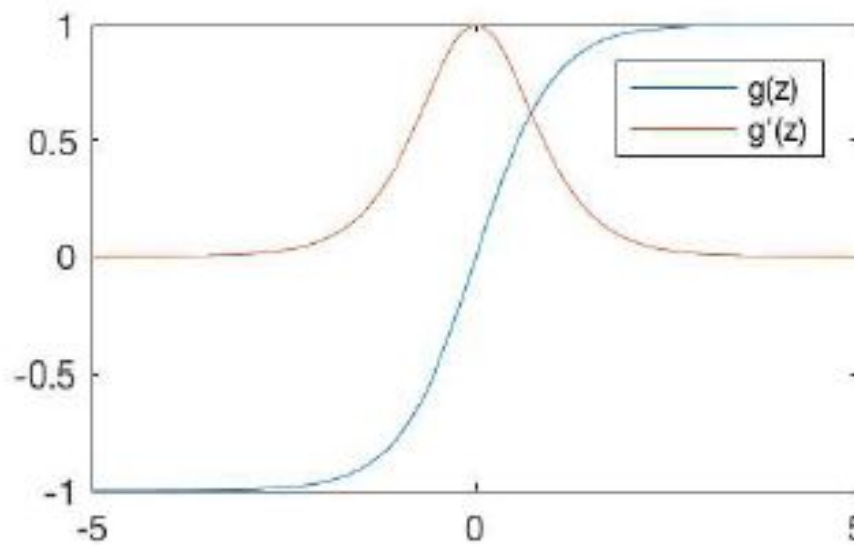


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

 `tf.nn.sigmoid(z)`

Hyperbolic Tangent

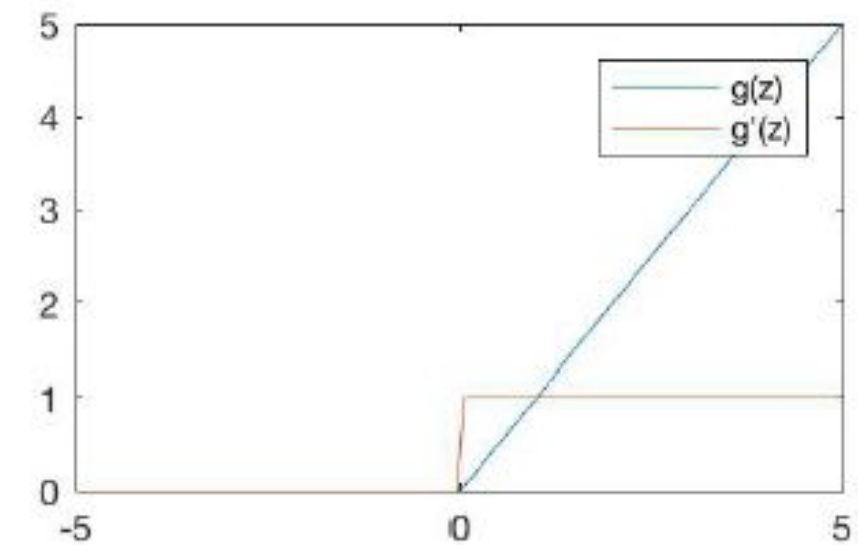


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

 `tf.nn.tanh(z)`

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

 `tf.nn.relu(z)`

Funciones de Activación en Scikit-Learn

En Scikit-Learn, la función de activación se define mediante el parámetro **activation** en el constructor del modelo. Este parámetro se aplica exclusivamente a las capas ocultas.

- **'relu'** (Predeterminada): $f(x) = \max(0, x)$. Es la opción por defecto. Rápida y eficiente para la mayoría de los problemas.
- **'logistic'**: Función Sigmoide $f(x) = \frac{1}{1+e^{-x}}$. Mapea valores al intervalo $(0, 1)$. Usada principalmente en redes pequeñas o con fines académicos.
- **'tanh'**: Tangente hiperbólica $f(x) = \tanh(x)$. Mapea valores al intervalo $(-1, 1)$. Útil cuando los datos de entrada están centrados en cero.
- **'identity'**: Activación lineal $f(x) = x$. No aplica transformación no lineal (convierte el bloque oculto en una transformación matricial lineal).

En Scikit-Learn **no puedes cambiar manualmente la función de activación de la capa de salida. El modelo la asigna automáticamente** según la tarea:

- **MLPClassifier**: Aplica **Sigmoide** (si es clasificación binaria) o **Softmax** (si es multiclase) en la salida.
- **MLPRegressor**: Aplica la función **Lineal** en la salida.

MLP es una arquitectura

- Una MLP es una arquitectura que puede utilizarse para diferentes tipos de problemas.

- **Regresión**

```
from sklearn.neural_network import MLPRegressor
```

</> Python

```
modelo = MLPRegressor(  
    hidden_layer_sizes=(50,),  
    max_iter=1000,  
    random_state=40  
)
```

- **Clasificación**

</> Python

```
from sklearn.neural_network import MLPClassifier  
  
modelo = MLPClassifier(  
    hidden_layer_sizes=(50,),  
    activation='logistic',  
    solver='sgd',  
    max_iter=100  
)
```

Funciones de Pérdida (*Loss Functions*)

En Scikit-Learn **no se pasa la función de pérdida como un argumento configurable**. El modelo la selecciona automáticamente según la clase que instances y el tipo de etiqueta y:

- **Log-Loss / Cross-Entropy (Entropía Cruzada):**
 - Se activa automáticamente al usar **MLPClassifier**.
 - Calcula la pérdida logística para clasificación binaria o multiclase.
- **Squared Loss / MSE (Error Cuadrático Medio):**
 - Se activa automáticamente al usar **MLPRegressor**.
 - Mide el error cuadrático entre la predicción continua y el valor

Trabajo Práctico 3: Flujo de Trabajo



Preprocesamiento: Normalizar y Aplanar

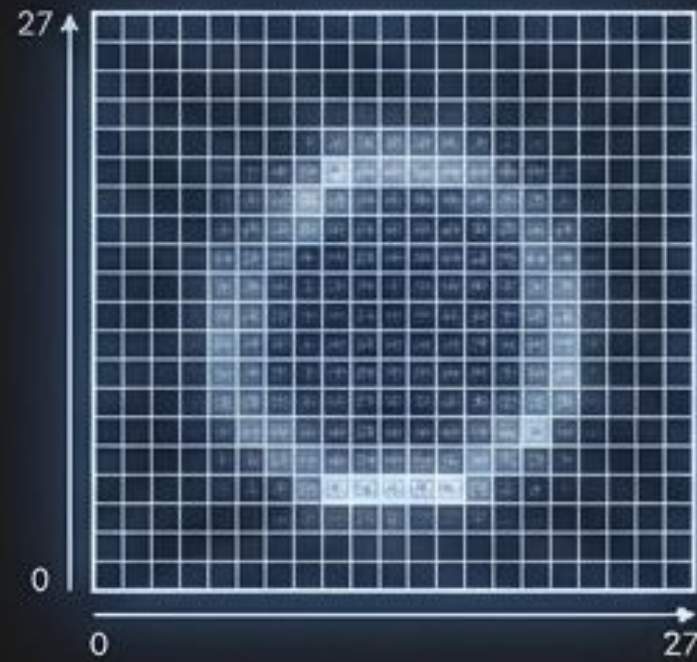


Imagen (2D Matrix)

`.reshape(len, -1)`



Vector de Características (1D Array - 784 inputs)

```
# 1. Normalizar (Escala de 0 a 1)
X_train = X_train / 255
X_test = X_test / 255

# 2. Aplanar (Imagen a Vector)
X_train = X_train.reshape(len(X_train), -1)
X_test = X_test.reshape(len(X_test), -1)
```

El Perceptrón no 've' imágenes cuadradas, solo procesa vectores numéricos.

Definiendo el Modelo en Python

PYTHON

SKLEARN

```
from sklearn.neural_network import MLPClassifier

# Definición de la Arquitectura
modelo = MLPClassifier(
    hidden_layer_sizes=(50,), # Una capa oculta con 50 neuronas
    activation='logistic',    # Función de activación Sigmoide
    solver='sgd',             # Optimizador: Descenso de Gradiente Estocástico
    max_iter=100              # Iteraciones máximas
)
```

Define la topología de la red.

Define la no-linealidad.

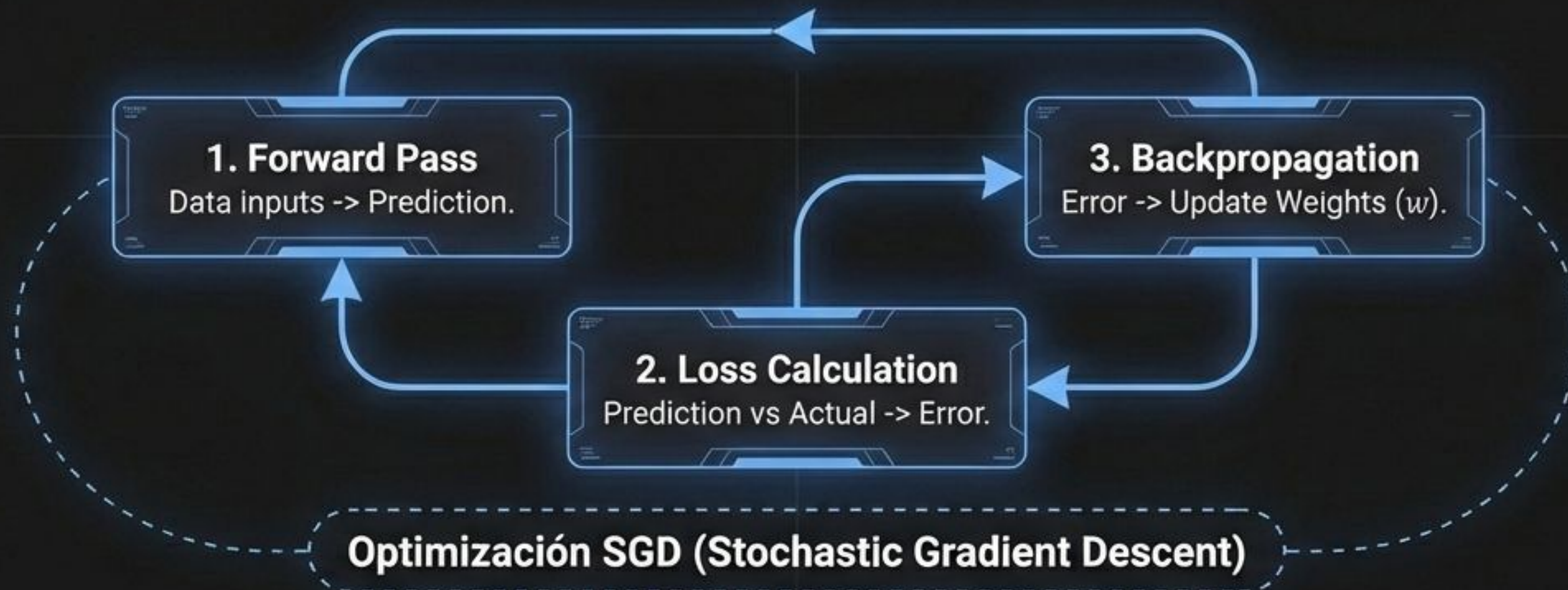
Define cómo aprende la red.

PYTHON

SKLEARN

El Entrenamiento: Método `.fit()`

```
modelo.fit(X_train, y_train)
```



Esta línea de código ejecuta el ciclo de aprendizaje iterativo para minimizar la función de pérdida.

Inspeccionando la "Caja Negra"

El modelo entrenado contiene propiedades matemáticas accesibles.

PYTHON

```
# Verificar estructura
print(modelo.n_layers_) # Número de capas
print(modelo.n_outputs_) # Neuronas de salida

# Inspeccionar matriz de pesos
print(modelo.coefs_[0].shape)
```

Matriz de Pesos (Weights)

0.345	-0.23	0.11	0.18	...	-1.02	0.39	0.38	0.33
-0.98	0.04	0.76	0.72	...	-0.93	0.17	0.16	0.21
-1.11	-0.42	0.19	0.52	...	-0.48	0.14	0.92	0.59
-0.34	0.08	0.26	0.97	...	-0.83	0.25	0.17	0.73
...	...	...	...	...	...	...	...	...
0.43	0.09	0.11	0.26	...	-0.03	0.25	0.01	0.11
-0.86	-0.37	0.01	1.20	...	-0.42	0.15	0.34	0.18
0.29	-0.49	0.03	0.58	...	0.82	0.22	0.37	0.27
-0.98	0.04	0.76	0.76	...	-0.88	0.65	0.76	0.75

La "inteligencia" se almacena como arreglos de números.

Predicción y Evaluación

El modelo entrenado se utiliza para hacer predicciones y se mide su rendimiento con métricas precisas.

PYTHON

```
# 1. Generar predicciones con datos nuevos
y_pred = modelo.predict(X_test)

# 2. Calcular métrica de éxito (F1 Score)
from sklearn.metrics import f1_score
score = f1_score(y_test, y_pred, average="weighted")

print(f"F1 Score: {score}")
```

Conceptos Clave

Predict: Aplica los pesos aprendidos a datos nunca antes vistos.

F1 Score: Una media armónica entre precisión y recall. Ideal para clases desbalanceadas.

MLP: Ejemplo de Clasificación

```
from sklearn.neural_network import MLPClassifier

# Scikit-Learn aplica ReLU en capas ocultas
# y asigna automáticamente Softmax + Cross-Entropy en la salida
mlp_clasificador = MLPClassifier(
    hidden_layer_sizes=(128, 64), # Dos capas ocultas (128 y 64 neuronas)
    activation='relu',            # Activación para las capas ocultas ('relu', 'tanh')
    solver='adam',                # Algoritmo de optimización del error
    max_iter=50,
    random_state=42
)

# Al hacer fit, detecta las 10 clases de MNIST de forma implícita
mlp_clasificador.fit(X_train, y_train)

# Muestra la pérdida final alcanzada durante el entrenamiento
print(f"Pérdida final (Cross-Entropy): {mlp_clasificador.loss_}")
```

MLP: Ejemplo de Regresión

```
from sklearn.neural_network import MLPRegressor

# Scikit-Learn aplica tanh en la capa oculta
# y asigna automáticamente Salida Lineal + MSE (Error Cuadrático Medio)
mlp_regresor = MLPRegressor(
    hidden_layer_sizes=(50,),
    activation='tanh',           # Probamos activación tanh en la capa oculta
    solver='adam',
    max_iter=100,
    random_state=42
)

# fit para predecir un valor numérico continuo (ej. precio de vivienda)
mlp_regresor.fit(X_train_reg, y_train_reg)

# Muestra la pérdida final alcanzada (MSE)
print(f"Pérdida final (MSE): {mlp_regresor.loss_}")
```